LeslieDolot_JupyterCapstone TechDeck

April 2, 2026

1 CAPSTONE PROJECT - Food Panda Order Cancellation

Submitted by Leslie Anne D. Dolot

1.1 ## Outline

- Problem & Data Overview
- EDA Key Findings
- Feature Engineering Approach
- Model Selection & Methodology
- Results & Model Comparison
- Feature Importance & Explainability
- Limitations & Future Work

1.2 # Problem and Data Overview

Foodpanda's review of August 2024–2025 data shows a high order cancellation rate (>30%), signaling issues across customers, riders, or restaurant partners. These cancellations lead to lost revenue (no commissions, delivery fees, or promo returns), increased operating costs (rider time/fuel, customer support, payment reversals), and poorer customer experience (delays, uncertainty, refunds). As a result, customers may switch to competitors, restaurants face wasted preparation and reduced trust, and riders experience lost income and lower morale—potentially affecting retention and service quality.

Additionally, frequent cancellations damage brand trust, reduce repeat usage, and distort demand forecasting, leading to poorer operational decisions.

Problem statement: "Which orders are likely to be cancelled before fulfillment?"

MAIN OBJECTIVE:

By being able to predict which orders are likely to be cancelled, we can reduce the number of order cancellations in a year and avoid or reduce the negative impacts of cancelled orders. Once the management is able to identify this, they could trigger proactive interventions (discounts, faster dispatch, alerts), provide changes, solutions and remedies before it results in a huge negative impact in the whole business financials.

SUCCESS METRICS:

Recall F1 Score Precision ROC-AUC and PR-AUC

For this case, I used Classification algorithm because the target variable "delivery_status" is a discrete categorical label. Since this is not a numerical value that has an increasing or decreasing range, then regression model will not work.

1.3 Data Overview

[1]:
```
import pandas as pd
file_name = 'Foodpanda Analysis Dataset.csv'
print('\n' + '='*80)
print('STEP 2: UPLOAD CSV FILE')
print('='*80)
df = pd.read_csv(file_name)
print(f'Loaded file: {file_name}')
print(f'Dataset shape: {df.shape}')
display(df.head())
```

```
================================================================================
STEP 2: UPLOAD CSV FILE
================================================================================
Loaded file: Foodpanda Analysis Dataset.csv
Dataset shape: (6000, 21)

  customer_id  gender     age       city signup_date order_id order_date  \
0       C5221    Male  Senior     Lahore   10/3/2023    O9221  1/10/2024
1       C2831    Male   Adult     Multan    7/7/2024    O6831  8/23/2023
2       C2851   Other  Senior     Multan   6/20/2025    O6851  8/23/2023
3       C1694  Female  Senior   Peshawar    9/5/2023    O5694  8/23/2023
4       C5641   Other  Senior  Islamabad  10/13/2023    O9641  3/27/2024

  restaurant_name dish_name category  …    price  payment_method  \
0      McDonald's    Burger  Italian  …  1291.14            Card
1             KFC    Burger  Italian  …   956.04          Wallet
2       Pizza Hut     Fries  Italian  …   882.51            Cash
3          Subway     Pizza  Dessert  …   231.30            Card
4       Pizza Hut    Burger  Chinese  …   620.96            Card

  order_frequency  last_order_date loyalty_points   churned rating  \
0               7        8/21/2025             42  Inactive      3
1              24       11/25/2024             81    Active      2
2              42        5/10/2025             82  Inactive      3
3              27        7/24/2025             45  Inactive      2
4              17        8/21/2025            206    Active      2

   rating_date delivery_status Unnamed: 20
0   11/29/2024       Cancelled         NaN
1    8/21/2025         Delayed         NaN
2    9/19/2024         Delayed         NaN
3    6/29/2025         Delayed         NaN
```

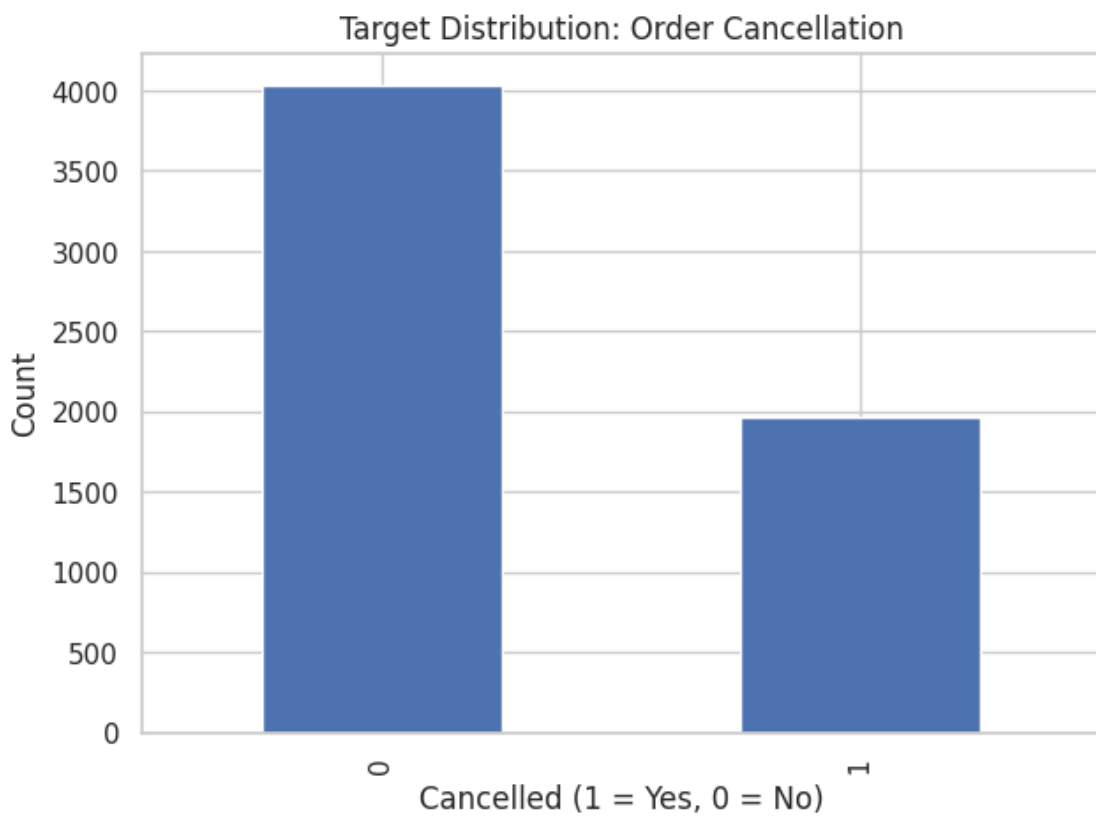
```
4     4/9/2025       Cancelled         NaN

[5 rows x 21 columns]
```

1.4 ## EDA Key Findings

- There are 32.32% cancelled orders in the past year from August 2024 to 2025, or half of the delivered orders are cancelled orders.
- The order cost of cancellations ranges from 100 to 1500 Indian Rupees.
- The months of January, March, August, September and October have the highest cancellations (>500 counts)
- The top 3 most influential features for order cancellation are the week of the year, order quantity, and a certain city in India.

1.5 We lost 1,571,276.37 Indian Rupees which is equivalent to $ 16,623.60 due to order cancellations.



1.6 EDA Key Findings

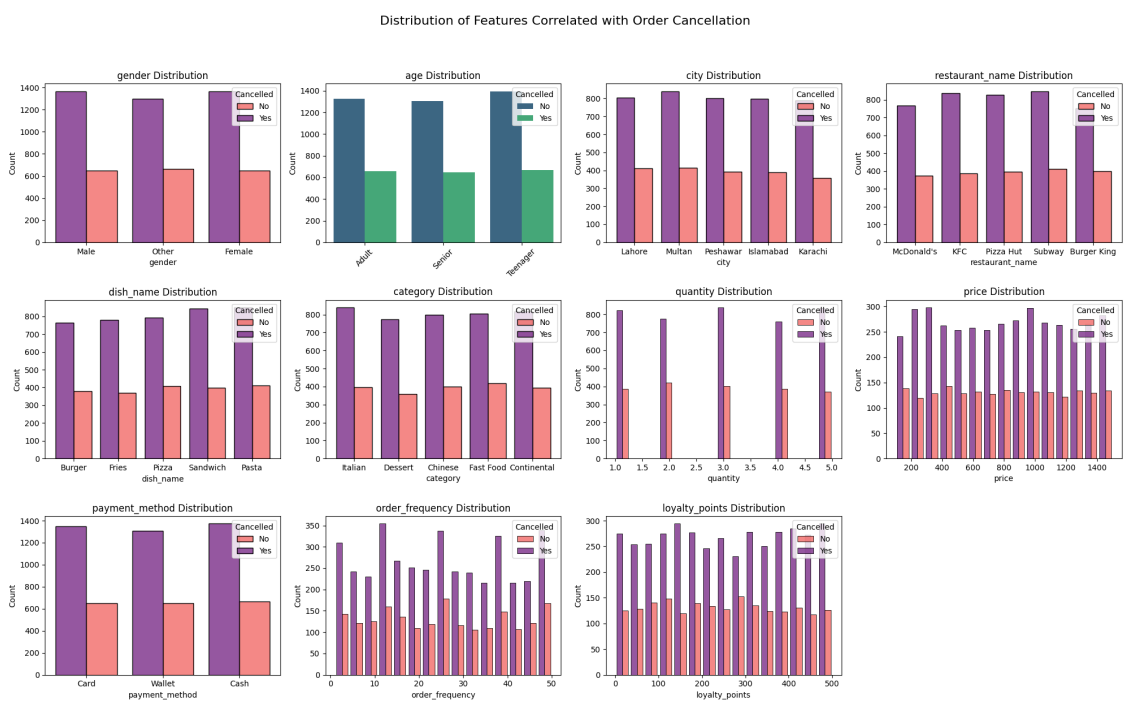
[28]:
```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```python
from sklearn.pipeline import Pipeline
import matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset
df = pd.read_csv('Foodpanda Analysis Dataset.csv')
# Drop irrelevant columns for predicting order cancellation
df_processed = df.drop(columns=['customer_id', 'order_id', 'signup_date',␣
↪'last_order_date', 'rating_date', 'rating', 'churned', 'Unnamed: 20'])
# Convert 'delivery_status' into a binary target variable 'is_cancelled'
df_processed['is_cancelled'] = (df_processed['delivery_status'] == 'Cancelled').
↪astype(int)
# Drop the original 'delivery_status' column as it's replaced by 'is_cancelled'
df_processed = df_processed.drop(columns=['delivery_status'])
# 'age' column processing:
df_processed['age'] = df_processed['age'].astype('category')
# Separate target variable
X = df_processed.drop('is_cancelled', axis=1)
y = df_processed['is_cancelled']
# Identify numerical and categorical columns
numerical_cols = X.select_dtypes(include=np.number).columns.tolist()
categorical_cols = X.select_dtypes(include='category').columns.tolist()
# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore')
# Create a column transformer to apply different transformations to different␣
↪columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])
# Create a pipeline that first preprocesses and then applies a model
pipeline = Pipeline(steps=[('preprocessor', preprocessor)])
# Fit and transform the data
X_processed = pipeline.fit_transform(X)
# Get feature names after one-hot encoding for categorical features
onehot_feature_names = pipeline.named_steps['preprocessor'].
↪named_transformers_['cat'].get_feature_names_out(categorical_cols)
# Combine numerical and one-hot encoded feature names
all_feature_names = numerical_cols + list(onehot_feature_names)
# Convert the processed data back to a DataFrame for easier inspection
X_processed_df = pd.DataFrame(X_processed, columns=all_feature_names)
# Separate target variable
X = df_processed.drop('is_cancelled', axis=1)
y = df_processed['is_cancelled']
# Identify numerical and categorical columns
numerical_cols = X.select_dtypes(include=np.number).columns.tolist()
```

```
categorical_cols = X.select_dtypes(include='category').columns.tolist()
# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore')
# Create a column transformer to apply different transformations to different␣
↪columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])
# Create a pipeline that first preprocesses and then applies a model
pipeline = Pipeline(steps=[('preprocessor', preprocessor)])
# Fit and transform the data
X_processed = pipeline.fit_transform(X)
# Get feature names after one-hot encoding for categorical features
onehot_feature_names = pipeline.named_steps['preprocessor'].
↪named_transformers_['cat'].get_feature_names_out(categorical_cols)
# Combine numerical and one-hot encoded feature names
all_feature_names = numerical_cols + list(onehot_feature_names)
# Convert the processed data back to a DataFrame
X_processed_df = pd.DataFrame(X_processed, columns=all_feature_names)
X_processed_df.info()
# Identify features for plotting (excluding the target variable and processed/
↪dropped columns)
plot_features = [
    'gender', 'age', 'city', 'restaurant_name', 'dish_name', 'category',
    'quantity', 'price', 'payment_method', 'order_frequency', 'loyalty_points']
# Determine the number of plots and grid size
num_features = len(plot_features)
num_cols = 4
num_rows = (num_features + num_cols - 1) // num_cols
plt.figure(figsize=(num_cols * 5, num_rows * 4))
plt.suptitle('Distribution of Features Correlated with Order Cancellation', y=1.
↪02, fontsize=16)
for i, feature in enumerate(plot_features):
    ax = plt.subplot(num_rows, num_cols, i + 1)
    if df_processed[feature].dtype == 'object' or df_processed[feature].dtype␣
↪== 'category':
        # For categorical features, use countplot
        sns.countplot(x=feature, hue='is_cancelled', data=df_processed, ax=ax,␣
↪palette='viridis')
        ax.set_title(f'{feature} Distribution')
        ax.tick_params(axis='x', rotation=45) # Rotate x-labels for better␣
↪readability
        ax.set_xlabel('') # Hide x-label to prevent overlap
```

```
    else:
        # For numerical features, use histplot
        sns.histplot(x=feature, hue='is_cancelled', data=df_processed,␣
↪multiple='dodge', shrink=.8, bins=15, ax=ax, palette='magma')
        ax.set_title(f'{feature} Distribution')
    ax.legend(title='Cancelled', labels=['No', 'Yes'])
    ax.set_ylabel('Count')
plt.tight_layout(rect=[0, 0, 1, 0.98])
plt.show()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   quantity         6000 non-null   float64
 1   price            6000 non-null   float64
 2   order_frequency  6000 non-null   float64
 3   loyalty_points   6000 non-null   float64
 4   age_Adult        6000 non-null   float64
 5   age_Senior       6000 non-null   float64
 6   age_Teenager     6000 non-null   float64
dtypes: float64(7)
memory usage: 328.3 KB
```



1.7 EDA Findings:

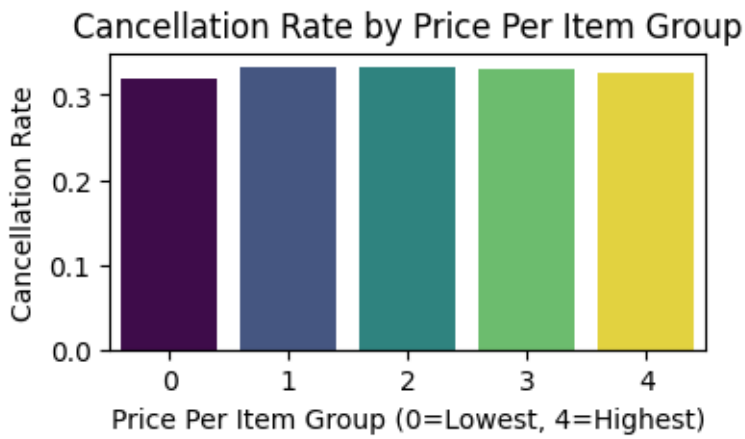
- There is no significant difference in cancellation of orders in the features such as gender, age, and payment method.
- City of Lahore and Multan are slightly higher than the rest of the 3 cities.
- Restaurants such as Subway, KFC and Pizza Hut have slightly higher order cancellations.
- Italian food has higher order cancellation.
- I terms of price, orders at the price range of 200 - 300 has the highest cancellation.

1.8 # FEATURE ENGINEERING APPROACH

- Dropping of identifiers such as customer id and order id since they are not related or helpful in determining order cancellations
- Dropping of data leakages such as rating, rating date, and churned since in the case of "rating", if an order is cancelled, then the rating could either be none or very low. This may result to the model learning that the low rating could yield to cancellation of orders and not the other way around.
- Review of potentially outlier variables were also done
- Z-Score normalization of numerical data to standardize the values and such that the model will not detect high value numbers as having more impact than those with lower values
- One-Hot-Encoding of categorical data
- Removal of data issues: some last order date occured after the order date
- Used delivery status (cancelled vs delivered) as the feature to compare with the other features
- Tested combining "restaurant" and "category" to see if this will simplify the features and provide better correlation, but this test failed.
- Tested combining the "price" and "quantity" of items ordered to determine the price per item and then correlated with the cancelled orders. The result showed almost equal distribution, hence this data transformation did not provide a correlation or pattern.

[24]:
```
import pandas as pd
file_name = '/content/Foodpanda Analysis Dataset.csv'
df_clean = df.copy()
df_processed = df.drop(columns=['customer_id', 'order_id', 'signup_date',␣
↪'order_date', 'last_order_date', 'rating_date', 'Unnamed: 20'])
df_processed['is_cancelled'] = (df_processed['delivery_status'] == 'Cancelled').
↪astype(int)
df_processed = df_processed.drop(columns=['delivery_status'])
df_processed['age'] = df_processed['age'].astype('category')
eda_df = df_processed.copy()
# Create 'price_per_item' feature
eda_df['price_per_item'] = eda_df['price'] / eda_df['quantity']
# Group 'price_per_item' into 5 bins
# Using qcut to create groups with approximately equal numbers of observations
eda_df['price_per_item_group'] = pd.qcut(eda_df['price_per_item'], q=5,␣
↪labels=False, duplicates='drop')
# Calculate cancellation rate per group, using the correct column name␣
↪'is_cancelled'
```

```
cancellation_rate_by_price_per_item = eda_df.
 ↪groupby('price_per_item_group')['is_cancelled'].mean().reset_index()
# Plotting the distribution of price per item with cancelled orders
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(4, 2))
sns.barplot(x='price_per_item_group', y='is_cancelled',␣
 ↪data=cancellation_rate_by_price_per_item, palette='viridis',␣
 ↪hue='price_per_item_group', legend=False)
plt.title('Cancellation Rate by Price Per Item Group')
plt.xlabel('Price Per Item Group (0=Lowest, 4=Highest)')
plt.ylabel('Cancellation Rate')
plt.show()
```



1.9 # MODEL SELECTION AND METHODOLOGY

I used Supervised Classification Algorithm for this scenario. Three algorithms were tested and compared:

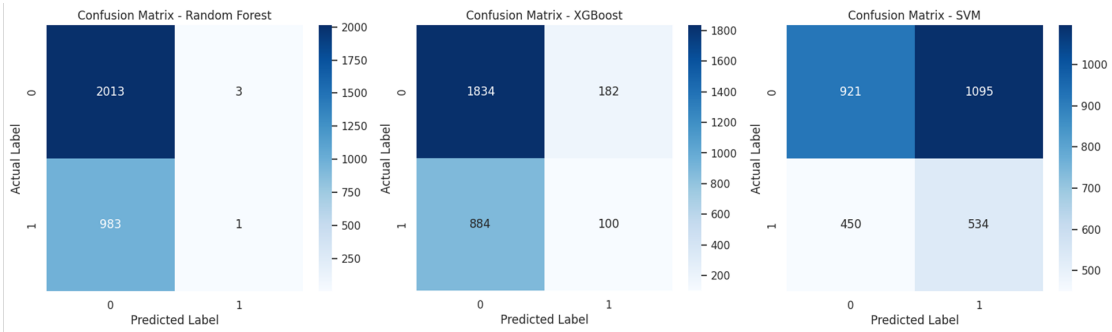
- Random Forest
- XGBoost
- SVM

Confusion Matrices were compared and analyzed during the process. The resulting Recall, F1 Score, Precision, and ROC-AUC and PR-AUC were compared between the three algorithms to determine which is the most appropriate model to use.

1.10 # Results & Model Comparison

- Random Forest: High accuracy but failed to detect cancellations (1/984 detected)
- XGBoost: High precision, but still missed most cancellations (100/984 detected) ## SVM: Best at detecting cancellations (>50% recall), despite it has more false positives

- Trade-off: SVM enables intervention but may increase costs due to unnecessary actions. Nevertheless, this will be the recommended model to achieve the objective of reducing the number of cancelled orders.

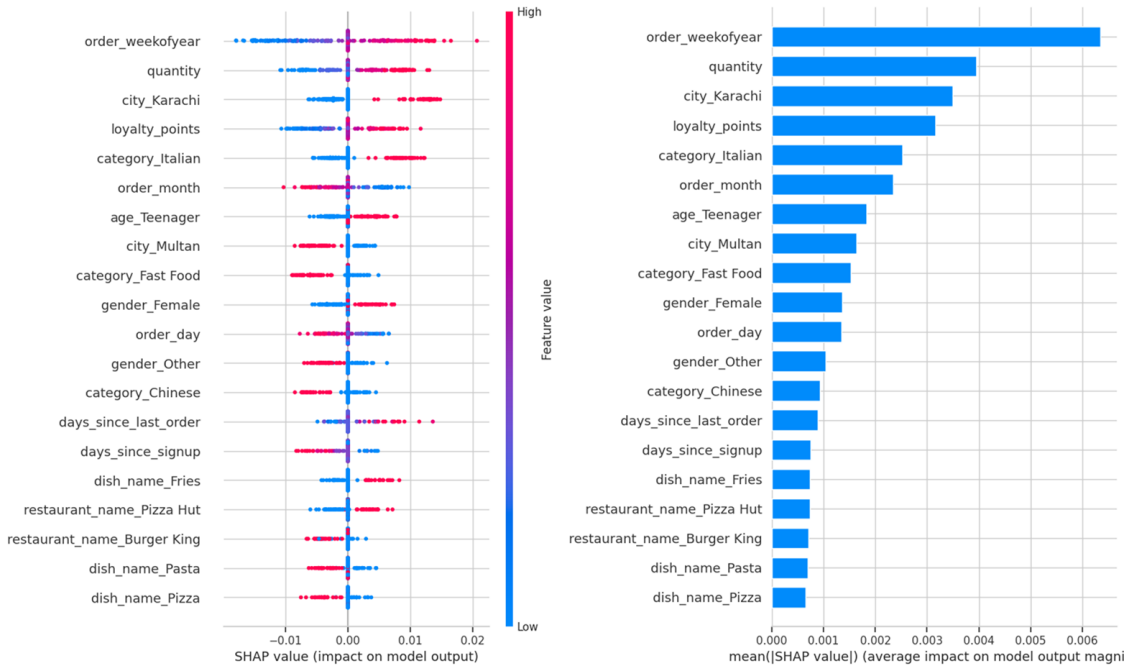


Model performance summary:

	model	accuracy	roc_auc	pr_auc	precision	recall	f1_score	correct_predictions	total_predictions
0	Random Forest	0.671333	0.520421	0.340028	0.250000	0.001016	0.002024	2014	3000
1	XGBoost	0.644667	0.508862	0.342794	0.354610	0.101626	0.157978	1934	3000
2	SVM	0.485000	0.501281	0.322063	0.327808	0.542683	0.408726	1455	3000

1.11 # FEATURE IMPORTANCE & EXPLAINABILITY

- The SVM model plot shows how each feature influences cancellation predictions
- Features are ranked by importance (top = most influential)
- Top drivers: order_weekofyear, quantity, city_Karachi
- X-axis (impact):
  - Right (+) → increases likelihood of cancellation (Class 1)
  - Left (–) → decreases likelihood (Class 0)
- Color meaning:
  - Red = higher feature values
  - More red points on the right → higher chance of cancellation



1.12 # LIMITATIONS & FUTURE WORK

- Results after use of fairness matrix such as demographic parity and equalized odds
  - there is significant disparity in predicted cancellation rates across different cities
  - there's a trend where the 'Other' group has the highest TPR and FPR, followed by 'Male' and then 'Female'.
- Result from reweighting test tells us that the current dataset is geographically skewed, and that skew could make the model unfair to users in underrepresented cities — either over-predicting or under-predicting cancellations for them. The moderate weight range (7–15) means we still have time to correct this through proper stratification and weighted training before it becomes a real fairness problem in the execution process. Once we are able to improve on this, then we can re-run the classification algorithms and optimize the model.
- Biases: Proposed mitigations
  - Recommend to do reweighting for city and gender.
  - Recommend to collect more data on the cities

1.13 Future Works

Missing Key Variables

The dataset lacks important factors that influence order cancellations, such as:

- Delivery time (estimated vs. actual)
- Order-to-delivery/cancellation time
- Driver assignment time
- Restaurant preparation time
- Cancellation reasons and types
- External factors (weather, traffic, holidays)

Missing context likely explains the low model accuracy and weak feature/SHAP importance

Data Coverage Limitation Data spans Aug 2024 – Aug 2025, but patterns vary by month Some months show higher/lower cancellations

A complete and consistent full-year dataset is needed to capture:

Seasonality (holidays, events) External disruptions (traffic, roadworks, etc.)

Recommendation Enrich dataset with operational and external variables Ensure more than a full-year coverage to improve model performance and reliability Enrich the dataset with users from underrepresented cities to fix the skew of data.